

SHIP ASSESSMENT



AI Workflow Ops Backend

First Stage Selection Task - SDE / AI Systems / Full-Stack

Build a mini AI-powered customer request routing system.

Auth + database + async AI classification + realtime admin updates + simple dashboard.

Time limit: 48 hours | Deliverable: repo + demo + README + architecture diagram

cognifyr.co

Internship Test: AI Workflow Ops Backend

A practical 48-hour assessment to separate real builders from resume polishers. Candidates must build a focused backend-heavy product that receives customer requests, routes them through an AI workflow, assigns priority, tracks status, and exposes realtime updates to an admin dashboard.

This is not a UI-only assignment. We are evaluating backend architecture, database design, async processing, AI workflow thinking, reliability, security basics, and clear technical explanation.

Assessment Type	First stage internship selection task
Target Roles	SDE, AI Systems, Backend, Automation, Full-Stack
Time Limit	48 hours
Expected Output	Working MVP with repo, demo/video, README, API docs, architecture diagram, and tradeoff note
Main Skill Signal	Can the candidate design and ship a real workflow system under constraints?

Candidate Brief

Build a mini AI-powered customer request routing system. The app should let a business receive customer requests, classify them using AI or a mock AI workflow, assign priority, track status, and show realtime updates in a simple admin dashboard.

The system should feel like a small internal operations tool that could actually be used by a support, sales, or automation team.

Scenario

Cognifyr receives customer messages from multiple channels such as website forms, WhatsApp, and email. Each request must be stored, classified, prioritized, routed into a workflow, and tracked by admins until it is resolved.

- 1 A customer message enters the system through an API or webhook simulation.
- 2 The request is saved immediately and marked as queued/new.
- 3 A background worker classifies the request using AI or a mock AI service.
- 4 The system stores the AI classification separately from the original request.
- 5 The admin dashboard receives a live update and shows the latest status.
- 6 An admin or agent can update status and add internal notes.

Required Build Scope

1	Backend API User/admin auth, create request, list/filter requests, view details, update status, add internal notes. REST API is required.
2	Database Design Design tables/collections for users, customer_requests, request_events, ai_classifications, and internal_notes. Explain schema choices.
3	AI Classification Workflow When a request is created, classify category, assign priority, generate a short summary, and store AI output separately. Mock AI is acceptable.
4	Async / Background Processing AI classification must not block the request creation API. Use BullMQ + Redis, a worker, Celery, or a clearly explained queue simulation.
5	Realtime Updates Admin dashboard should receive live updates using WebSocket, Socket.io, Server-Sent Events, Supabase realtime, Firebase, or equivalent.
6	Security JWT/session auth, password hashing, validation, rate limiting or abuse protection, environment variables, and no secrets in the frontend or repository.
7	Frontend Dashboard Simple but usable admin dashboard with login, request list, request detail, filters, notes, status update, and realtime status/classification changes.
8	Deployment Deploy on Vercel, Render, Railway, Fly.io, Cloudflare, Supabase, Firebase, VPS, or provide a clear demo video with local setup.

MVP Principle

A clean, working MVP is better than an unfinished enterprise system. The project should prove that the candidate can design workflows, write maintainable backend code, and explain technical tradeoffs clearly.

AI Workflow and Skills Layer

The AI layer should be designed like a replaceable skill/module. Candidates can use a real provider if they have access, but the assessment must be possible with a clean mock provider.

Classification Skill	Classifies the request into sales, support, urgent, spam, or other. It should return structured output, not random text.
Priority Skill	Assigns low, medium, or high priority based on message urgency, intent, customer language, and business impact.
Summary Skill	Generates a short internal summary that helps an admin understand the request quickly.
Routing Skill	Decides which queue or workflow should handle the request. This can be simple, but it should be clearly explained.

Expected AI Output Shape

```
<font name="DejaVuMono">{ "category": "support", "priority": "high", "summary":  
"Customer cannot access paid dashboard after payment.", "confidence": 0.86, "reason":  
"Payment + login issue requires fast support response." }</font>
```

Provider / Tooling Bonus

- OpenAI or Gemini integration for classification and summarization.
- Claude-style reasoning adapter for long-form analysis, routing justification, or architecture review.
- Codex-style coding/task adapter if the candidate extends the system into an AI skill orchestrator.
- n8n, LangChain, local LLM, or clean mock provider abstraction.
- Prompt injection safety note explaining how user-provided messages are treated as untrusted input.

Important: Real API access is not required. A well-designed mock AI layer is better than a messy hardcoded API call.

API and Data Model Expectations

The exact implementation is flexible, but the candidate should expose a clean REST API and maintain a traceable workflow history.

Recommended REST Endpoints

Method	Endpoint	Purpose
POST	/auth/login	Login and receive session/JWT
POST	/requests	Create a customer request and enqueue AI classification
GET	/requests	List/filter requests by status, priority, category
GET	/requests/:id	Fetch request detail, AI classification, notes, event timeline
PATCH	/requests/:id/status	Update request status
POST	/requests/:id/notes	Add internal note
POST	/webhooks/inbound	Optional webhook simulation for WhatsApp/email/website
POST	/requests/:id/retry-classification	Optional retry for failed AI classification

Minimum Data Model

- users: admin/agent accounts, password hash, role, created timestamp.
- customer_requests: original message, source channel, customer info, status, category snapshot, priority snapshot.
- ai_classifications: request_id, provider, category, priority, summary, confidence, raw output, error state.
- request_events: request_id, event type, old/new values, actor, metadata, timestamp.
- internal_notes: request_id, author, note body, created timestamp.

Candidates must explain relationships, indexes, and why AI output is stored separately from the original customer request.

Expected Architecture Flow

The system should show a clear separation between API layer, database layer, background worker, AI provider/mock layer, realtime service, and admin UI.

Step	Stage	Expected Behavior
1	Customer submits request	Website form, WhatsApp/email webhook simulation, or direct API call.
2	API stores request	Request is saved immediately. Response should not wait for AI classification.
3	Job is queued	Worker receives request ID and starts classification.
4	AI/mock provider runs	Classifier returns category, priority, summary, confidence, and reason.
5	Database is updated	AI output, request status, and event timeline are persisted.
6	Realtime event emitted	Dashboard receives live update through WebSocket/SSE/realtime platform.
7	Admin acts	Admin filters, reviews, updates status, and adds internal notes.

Frontend Requirements

- Login page for admin access.
- Request list page with filters for priority, category, and status.
- Request detail page showing original message, AI classification, notes, and event timeline.
- Status update action and internal notes section.
- Realtime update indicator or clearly visible live refresh behavior.

UI quality note: The UI does not need to be beautiful. It must be usable, understandable, and connected to the backend correctly.

Security, Reliability, and Delivery Rules

Required Security Basics

- JWT or session authentication for protected admin APIs.
- Password hashing using bcrypt, argon2, or platform-auth equivalent.
- Input validation for request creation, login, filters, status updates, and notes.
- Rate limiting or basic abuse protection on public endpoints.
- Environment variables for secrets and provider API keys.
- No API keys, tokens, passwords, or private data committed to GitHub.
- Basic explanation of webhook/API signature validation for inbound channels.

Reliability Expectations

- Failed AI classification should not break request creation.
- The system should track failed jobs and expose status clearly.
- Retry logic is a bonus but failure states should still be visible.
- Events should be recorded so reviewers can understand what happened and when.

Submission Deliverables

- 1 GitHub repository link with clean commit history.
- 2 Live demo link or 3-5 minute demo video.
- 3 README with setup instructions, environment variables, architecture explanation, schema explanation, AI workflow explanation, and known limitations.
- 4 API documentation.
- 5 Architecture diagram.
- 6 Short tradeoff note: "What I would improve with two more weeks."

Important Rules

- Do not copy-paste a complete project from the internet.
- Using AI tools is allowed, but the candidate must understand and explain the architecture.
- Real AI API usage is optional. Mock AI is acceptable if the architecture is clean.
- Keep the project focused. A clean working system is better than an unfinished overbuilt system.

Evaluation Rubric

Area	Score	What We Look For
Backend/API Design	25%	Clean REST APIs, request lifecycle, auth, error handling, maintainability.
Database/Schema Thinking	15%	Good schema, relationships, event history, indexes, and explanation.
AI Workflow Integration	15%	Classification flow, mock/real AI abstraction, structured output, clean storage.
Async + Realtime Architecture	15%	Background processing, queue thinking, live updates, failure handling.
Security/Reliability	10%	Auth, validation, hashing, rate limiting, secret handling, safe webhook thinking.
Code Quality + README	10%	Readable code, setup clarity, clean commits, useful docs.
Product Usability	10%	Usable dashboard, filters, detail page, understandable workflow.

Hard Bonus Challenges

- Webhook endpoint for WhatsApp/email simulation.
- Idempotency key for duplicate request prevention.
- Audit log / request event timeline.
- Retry logic for failed AI classification.
- Docker Compose for API + DB + Redis.
- Tests for core backend routes.
- Role-based access: admin vs agent.
- Prompt injection safety note.
- Provider health checks and fallback logic.
- Cost/latency estimation per AI provider.
- Claude/Codex/Gemini/local provider adapter pattern.

Reviewer Notes and Candidate Short Prompt

What This Test Reveals

- Whether the candidate can build more than basic CRUD.
- Whether they understand async workflows and realtime systems.
- Whether they can safely integrate AI or design a clean mock layer.
- Whether they can think in data models, events, statuses, and failure states.
- Whether they can explain architecture, tradeoffs, and limitations clearly.
- Whether they can ship a usable end-to-end MVP under time pressure.

Candidate Short Version

Build a mini AI-powered customer request routing system with auth, database, async AI classification, realtime admin updates, and a simple dashboard. Submit repo, demo, README, architecture diagram, API documentation, and tradeoff notes within 48 hours.

Recommended Sending Note

This assessment is intentionally practical. We do not expect a perfect enterprise system. We are looking for clarity, ownership, working software, and technical judgment. Anyone who completes 70% cleanly is worth serious consideration.



Cognifyr
AI Systems, Automation, and Product Engineering
Website: cognifyr.co